Tarun sharma

20/01/2026

Microservices Mastery with Node.js & NestJS

Welcome to Microservices Mastery

Unleashing Modern Backend Power

Microservices enable scalable, flexible architectures

Why Tech Giants Choose Microservices

Netflix and Amazon rely on distributed systems for growth

Hands-On, Architecture-First Approach

Learn by building real-world, production-ready systems

Why Microservices Matter

Monoliths vs. Microservices

- Monoliths are single, unified codebases; microservices split functionality into independent services.
- Microservices enable independent deployment and scaling.

Scalability and Resilience

- Easily scale individual services based on demand.
- Failures are isolated, improving overall system reliability.

Team Autonomy

- Teams own and deploy their services independently.

When to Choose Microservices

- Best for complex, rapidly growing applications or organizations with multiple teams.

Core Architecture Principles

Define Clear Service Boundaries

Use Domain-Driven Design to align services with business domains

Adopt the 12-Factor App Methodology

Ensure portability, scalability, and maintainability of services

Migrate from Monoliths with Proven Patterns

Apply strategies like the Strangler Fig pattern for gradual migration

Communication & Data Patterns

Service Communication Methods

Leverage REST, gRPC, and message queues for flexible interactions.

Event-Driven & Pub/Sub Models

Enable real-time, decoupled workflows with events and pub/sub.

Distributed Data Management

Adopt database-per-service, Sagas, CQRS, and event sourcing for consistency.

Building with Node.js & NestJS

Hands-On Microservices Development

Build real projects using Express and NestJS frameworks.

Diverse Communication Protocols

Implement microservices with TCP, RabbitMQ, gRPC, and Redis.

Project Structure & Code Organization

Adopt best practices for scalable, maintainable codebases.

Resilience, Security, and DevOps

Reliability Patterns for Robust Services

- Implement circuit breaker, retry, and bulkhead to handle failures gracefully

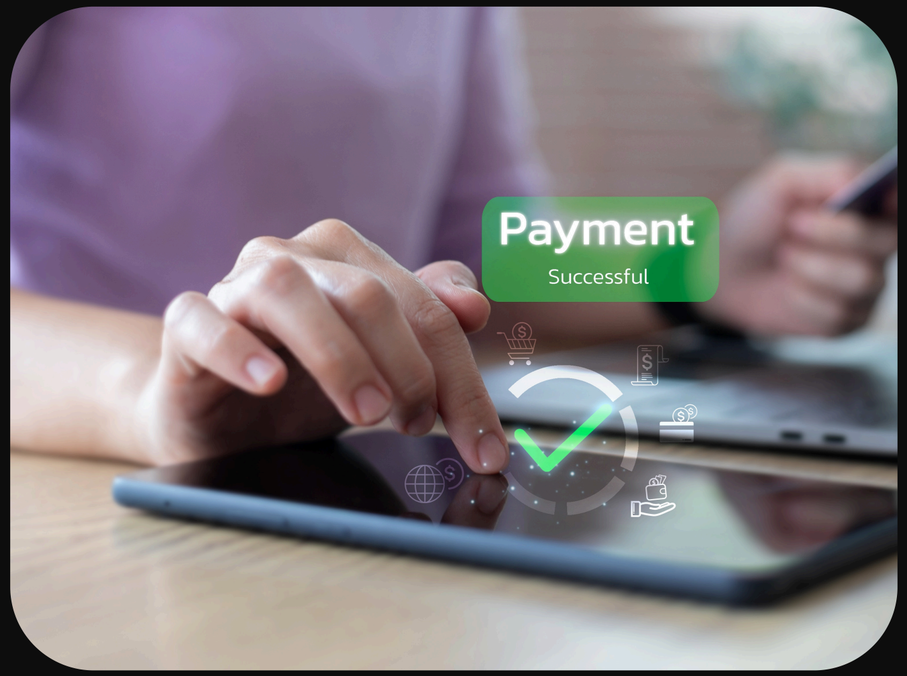
Securing Microservices End-to-End

- Protect APIs with JWT, OAuth2, and API keys for authentication and authorization

Modern DevOps for Scalable Deployments

- Leverage Docker for containerization and Kubernetes for orchestration
- Automate builds and deployments with CI/CD pipelines

Real-World Use Cases



E-commerce and Payment Workflows

Design scalable shopping carts and secure payment flows

Order Management with Sagas

Coordinate distributed transactions for reliable order processing

Real-Time Notifications

Deliver instant updates to users with event-driven messaging

Analytics with CQRS

Separate read/write models for high-performance reporting

Production-Ready Reference Projects

Access practical demos to jumpstart your own solutions

Key Takeaways & Next Steps

Confidently Design Scalable Microservices

Apply architecture principles to real-world systems

Implement Proven Patterns and Best Practices

Leverage DDD, 12-Factor App, and resilience strategies

Continue Learning and Innovating

Experiment, build, and share your microservices expertise